

MedXpiryTracker

Medicine expiry tracking and reminder app for iOS.
Bundle identifier: `com.nixutils.Medicine-Date-Alerter`
Marketing version: 1.0
Minimum supported iOS: 26.5

1. Overview

MedXpiryTracker (internally codenamed *PillEye*) is a single-screen iOS application that helps users keep track of the manufacturing and expiry dates of medicines and household pharmaceuticals. The app schedules a local notification one day before each saved medicine's expiry date so the user can dispose of, refill, or use the item in time.

The user enters medicine details either by typing or by using the on-device camera to scan label text using Apple's Vision text recognition framework. All data is stored locally inside the app's sandbox; no servers, accounts, or analytics services are contacted.

2. Key Features

- **Manual entry** of medicine name, manufacturing date, expiry date, and snooze interval.
- **Camera-based text scanning** for medicine names and dates using `VisionKit` + `Vision`.
- **Local notifications** scheduled one day before expiry, with Snooze and Cancel actions.
- **Persistent on-device storage** as encrypted JSON inside Application Support.
- **Edit and delete** saved medicines via row swipe actions.
- **Light, friendly UI** with accessibility identifiers for UI tests.

3. App Architecture

The app is built with SwiftUI and Swift Concurrency (`async` / `await`). It is organized into a small number of single-responsibility files:

File	Responsibility
<code>Medicine_Date_AlerterApp.swift</code>	Application entry point and root scene.
<code>ContentView.swift</code>	Main form, saved list, popups, and notification routing handlers.
<code>Medicine.swift</code>	Medicine model, validation rules, and update DTO.
<code>MedicineStore.swift</code>	Observable store: load, save, update, delete, import/export to disk.
<code>NotificationScheduler.swift</code>	Wraps <code>UNUserNotificationCenter</code> for scheduling and cancellation.
<code>AppNotificationDelegate.swift</code>	Foreground notification presentation and tap routing.
<code>MedicineNotificationRoute.swift</code>	Bridges notification taps to the active view.
<code>TextScannerView.swift</code>	Camera viewfinder with on-device text recognition.
<code>MedicineDateParser.swift</code>	Parses scanned strings into <code>Date</code> values.
<code>MedicineNameParser.swift</code>	Selects the most likely medicine name from scanned text.
<code>SnoozeOption.swift</code>	Standard snooze intervals and label formatting.
<code>AlarmNotificationSound.swift</code>	Applies the urgency / sound on the reminder notification.
<code>DimensionalButtonStyle.swift</code>	Custom SwiftUI button style used app-wide.

4. User Flow

1. **Launch:** the app loads any previously saved medicines from disk on a background task.
2. **Add medicine:** the user enters or scans a medicine name.
3. **Set dates:** the user either picks the manufacturing/expiry date manually or scans the printed label.
4. **Choose snooze:** the user picks a snooze duration (used if they snooze the reminder).
5. **Save:** the app validates the form, persists the medicine to disk, and schedules a local notification.
6. **Reminder:** one day before expiry, iOS delivers a notification with Snooze and Cancel actions.
7. **Tap notification:** tapping opens the details popup with Delete and Cancel buttons.
8. **Edit or delete:** swipe a row in the saved list to edit snooze or delete the medicine.

5. Permissions

Permission	Why it is needed	Info.plist key
Camera	To scan the printed medicine name and dates on the package label using on-device Vision text recognition.	<code>NSCameraUsageDescription</code> : "Scan medicine labels and dates so you can edit and save expiry reminders."
Notifications	To deliver the expiry reminder one day before the medicine expires. Requested with alert, sound, and badge.	Requested at runtime via <code>UNUserNotificationCenter.requestAuthorization</code> .

6. Privacy

- The app does **not** collect or transmit any personal data.
- Scanned text is processed entirely on-device by Apple's Vision framework.
- Saved medicine records live in the app's private Application Support directory and are removed when the app is deleted.
- The data file is written with `FileProtectionType.complete`, so it is encrypted at rest and inaccessible while the device is locked.
- No third-party SDKs, no analytics, no advertising frameworks are linked into the app.

7. Data Storage

Medicines are persisted as ISO-8601-formatted, pretty-printed JSON at:

`~/Library/Application Support/PillEye/medicines.json` (within the app sandbox)

The store supports `exportData()` and `importData(_:)` for use in future backup or migration features. There is no fixed cap on the number of saved entries; the practical limit is the iOS 64-pending-notification budget per app.

8. Accessibility

- Standard SwiftUI controls provide VoiceOver labels out of the box.
- Key controls include accessibility identifiers (`medicineNameField`, `saveMedicineButton`, `validationMessage`) for automated UI tests.
- The interface uses Dynamic Type-friendly fonts and high-contrast palette colours.
- The app is locked to light mode for predictable contrast on coloured form rows.

9. Build and Distribution

Setting	Value
Bundle identifier	<code>com.nixutils.Medicine-Date-Alerter</code>
Display name	MedXpiryTracker
Marketing version	1.0
Deployment target	iOS 26.5
Target devices	iPhone and iPad (Universal)
Code signing	Automatic (development team BPH648379N)
Frameworks used	SwiftUI, Foundation, UserNotifications, VisionKit, Vision, AVFoundation

Note: the camera usage description shown in the App Store privacy section is taken from the value of `INFOPLIST_KEY_NSCameraUsageDescription` in the target's build settings.

10. How to Use the App

10.1 Add a medicine

1. Tap the medicine name field and type the name, or tap *Scan medicine name* and point the camera at the package.
2. Inside the scanner, tap the recognised text you want, then press *OK*.
3. For the manufacturing date, either tap *Set manually* to use the date picker, or tap *Scan* to read the printed date.
4. Repeat for the expiry date.
5. Pick a snooze duration. This is used when you tap *Snooze* on the reminder notification.
6. Tap *Save medicine*. If something is missing or the expiry date is not after the manufacturing date, the form will show an inline message.

10.2 React to a reminder

1. One day before the expiry date the device will show an "Expiry reminder" notification.
2. From the lock screen or notification center, you can tap *Snooze* to re-fire later, or *Cancel* to dismiss.
3. Tapping the notification body opens the app and shows the medicine details popup with Delete and Cancel actions.

10.3 Edit or delete a saved medicine

1. In the *Saved medicines* list, swipe a row from right to left.
2. Tap *Edit* to change the snooze interval, then tap *Save*. The notification is re-scheduled.
3. Tap *Delete* to remove the medicine. The scheduled notification is cancelled too.

11. Notifications - Behaviour Details

- The reminder fires at the same time of day as the moment the medicine was saved, one calendar day before `expiryDate`.
- If that moment has already passed when the medicine is saved, the reminder is scheduled one minute into the future so the user still sees it.
- Each notification carries the medicine's UUID and snooze interval in `userInfo`, never the medicine name or label text.
- The notification category `MEDICINE_EXPIRY_REMINDER` exposes two actions: `SNOOZE_MEDICINE_REMINDER` (snooze by the saved interval) and `CANCEL_MEDICINE_REMINDER` (mark dismissed).

12. App Store Submission Checklist

- App icon: 1024x1024 PNG provided at `Assets.xcassets/AppIcon.appiconset/AppIcon.png` with light, dark, and tinted variants.
- Display name: MedXpiryTracker.
- Camera privacy string: present.
- Notification authorization: requested only when the user saves the first medicine.
- No background modes, no entitlements beyond default.
- No third-party SDK disclosures required.
- Privacy nutrition labels: Data Not Collected.

13. Testing

- Unit tests live in `Medicine Date AlerterTests` and use the Swift Testing framework.
- UI tests live in `Medicine Date AlerterUITests` and use the XCUIAutomation framework.
- Tests can construct `MedicineStore` with a fake `NotificationScheduling` and `storageURL:nil` to stay isolated from real notifications and disk.